

SPE Final Project Report

RedBus: Microservices-Based Ticketing Platform with DevOps Automation

Course: CSE 816 - Software Production Engineering

Repository: [Rahul09123/Red-Bus](#)

1. Abstract

This project implements a production-style bus ticketing platform using a microservices architecture and an automated DevOps pipeline. The system is not only a functional full-stack application, but also an SDLC automation exercise where source-code updates can be built, packaged, pushed, and deployed with minimal manual intervention. The platform includes role-based business flows, Kubernetes-native deployment artifacts, centralized logging with ELK, and operations-focused concerns such as readiness/liveness probes and Horizontal Pod Autoscaling.

The final outcome demonstrates how software engineering decisions at application level and infrastructure level must work together. Instead of treating backend, frontend, CI/CD, and operations as separate pieces, this project integrates all of them into one coherent delivery model.

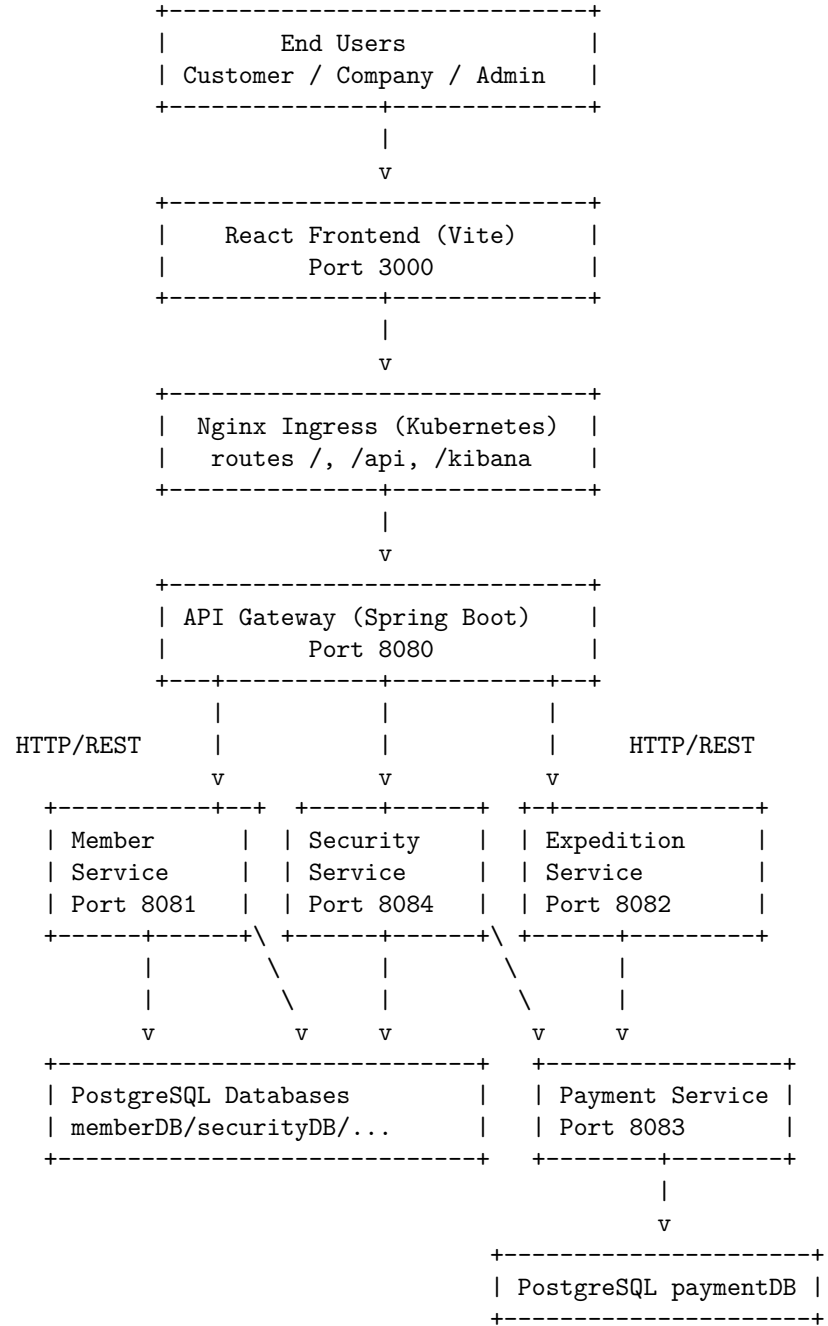
2. Problem Context and Objectives

A ticketing platform is a good case study for software production engineering because it naturally contains multiple independent business capabilities: identity, customer profile management, schedule/search, booking, and payment. In a monolithic design, these capabilities are tightly coupled and harder to evolve. This project addresses that limitation by splitting responsibilities into separate services, then adding deployment and monitoring automation around them.

The main objective was to build a realistic DevOps lifecycle around a domain-specific system. That means each change in the repository should flow through build and packaging pipelines, produce deployable containers, and be observable at runtime through centralized logs. The architecture was intentionally designed to show modularity, isolation, and scalability rather than only feature completeness.

3. System Architecture Overview

3.1 Architecture Diagram



Service Discovery and Ops Plane:

- Eureka Server: registration/discovery support
- ELK Stack: Filebeat -> Logstash -> Elasticsearch -> Kibana
- Vault + Kubernetes Secrets: secret-oriented runtime configuration

3.2 Architecture Explanation

The frontend is the user interaction layer and never calls internal microservices directly. All external traffic first enters through Ingress, which exposes path-based routes. API calls are forwarded to the API Gateway, and the gateway acts as the single entry point to domain services. This avoids exposing many internal services publicly and centralizes request handling.

Each backend service owns a focused business area and runs as an independent Kubernetes deployment. The services communicate over internal cluster networking using HTTP/REST. At runtime, logs from all pods are collected through Filebeat and sent through Logstash into Elasticsearch, with Kibana used for visualization and troubleshooting.

4. Service-by-Service Explanation

4.1 API Gateway Service

The API Gateway is the orchestration and routing layer for the backend. It receives frontend calls under `/api/...`, validates and transforms request/response models where required, and forwards calls to internal services such as member, security, expedition, and payment services. In this repository, the gateway has explicit service URL mappings to internal service DNS names and ports, which makes the communication paths transparent and easy to reason about.

From a production engineering perspective, this service simplifies client architecture because the frontend integrates with one endpoint instead of many. It also makes future concerns like centralized rate limiting, auth middleware, and API policy enforcement easier to add.

4.2 Eureka Server

Eureka Server provides service discovery support for Spring Cloud clients in the deployment topology. Services can register themselves and discover peer services through the registry model. Even when some direct internal URLs are configured, keeping Eureka in the architecture demonstrates distributed-system readiness and supports service lookup patterns that are common in microservice environments.

Operationally, Eureka helps reduce hard dependency on fixed instance addresses and supports more flexible scaling or restart scenarios.

4.3 Member Service

The member service handles user-domain entities and profile-centric functions. It is responsible for registration-related operations and role-profile information management. Business data handled here is persisted in the member domain database, keeping profile concerns isolated from booking or payment concerns.

It is called synchronously by the API Gateway over HTTP and returns structured responses to support frontend profile and account workflows.

4.4 Security Service

The security service manages authentication session lifecycle. It exposes endpoints such as session creation, logout, and session checks, and contains role-aware session logic for admin, company, and customer users. Instead of treating authentication as a simple token check, the implementation keeps explicit session state and validation rules, which is useful in role-sensitive systems.

The API Gateway consults this service for session validation and login/logout orchestration. This separation keeps auth concerns centralized and independent from feature services.

4.5 Expedition Service

The expedition service represents the transport domain layer. It handles expedition creation, listing, seat availability, ticket retrieval, and reservation-oriented workflows. This service directly models the core project domain, which is why it is central to customer and company user flows.

By isolating expedition logic from membership and auth, the project keeps domain boundaries clean. It can be scaled independently if search and booking traffic grows faster than other components.

4.6 Payment Service

The payment service is responsible for payment-related operations and persists its own payment-domain data. It is decoupled as a dedicated service so that payment validation, card handling, and transaction operations can evolve without forcing unrelated changes in expedition or member services.

In request flow terms, payment operations are triggered via the gateway as part of booking-associated sequences, and all responses are returned to clients through the same gateway path.

5. How Services Communicate

The communication model in this project is primarily synchronous HTTP/REST over Kubernetes internal networking:

1. The frontend sends requests to `/api/*` paths.
2. Ingress forwards those requests to API Gateway.
3. API Gateway maps each operation to the appropriate service endpoint:
 - `member-service:8081`
 - `expedition-service:8082`
 - `payment-service:8083`
 - `security-service:8084`
4. Each service executes domain logic and talks to its database schema/domain.
5. Responses return back through API Gateway to the frontend.

A second communication channel exists for operational telemetry: each pod writes logs, Filebeat ships logs to Logstash, Logstash indexes into Elasticsearch, and Kibana reads from Elasticsearch for monitoring and debugging.

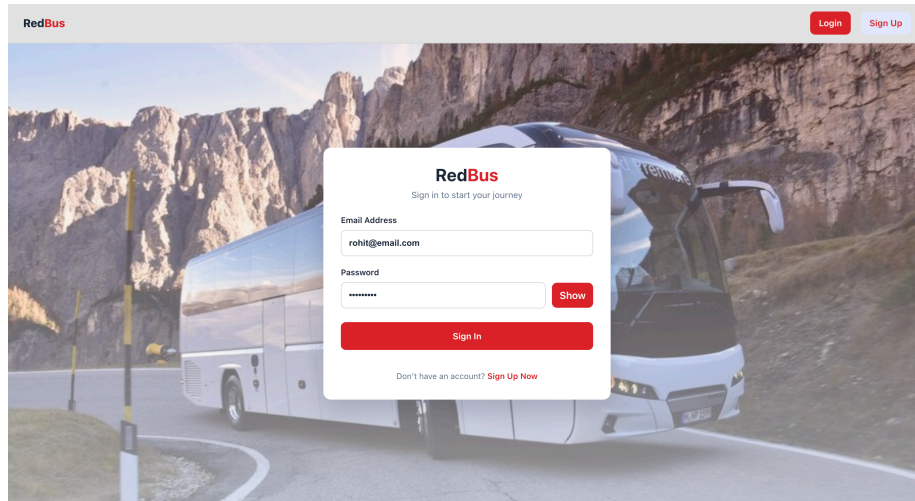
This dual-channel model (business traffic + observability traffic) is important in production systems because debugging distributed failures is as critical as serving functional requests.

6. Frontend Architecture and User Flow

The React frontend uses route guards and role-specific layouts to control user experience by identity type. Authentication pages are separated from protected areas, and domain pages are split into customer, company, and admin flows. This structure mirrors backend role boundaries and prevents route-level cross-access in normal operation.

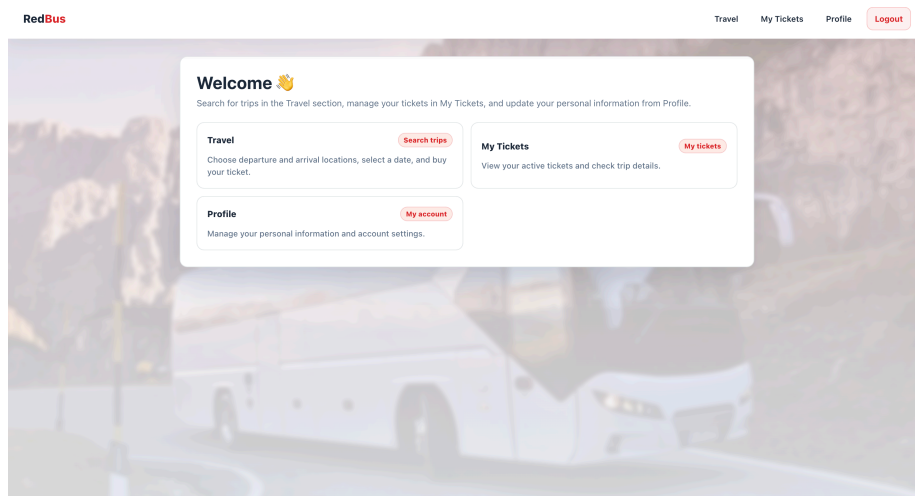
Customer users access travel search, booking, profile, and ticket pages. Company users access expedition management screens. Admin users access verification and approval pages. This alignment between UI modules and backend service capabilities improves maintainability and reduces accidental coupling.

Frontend Evidence 1



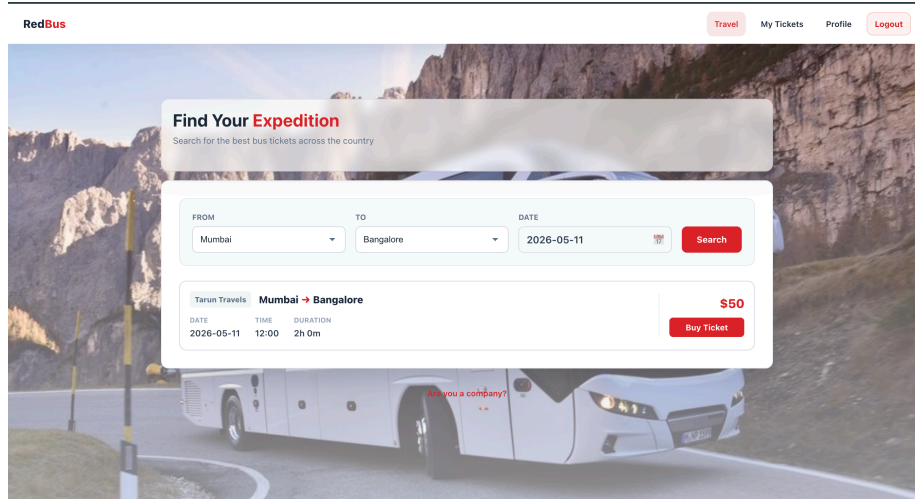
This screen shows an initial user-facing page of the platform and demonstrates the baseline visual structure used in the frontend module.

Frontend Evidence 2



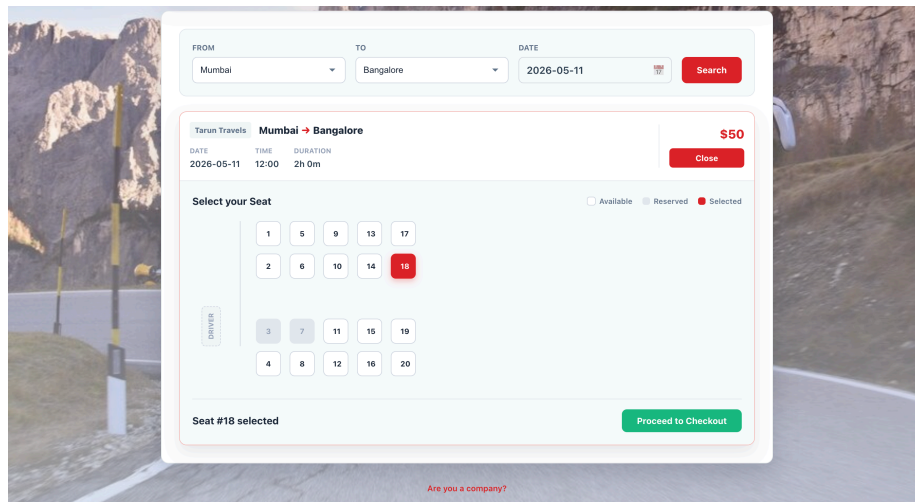
This view reflects continuity of the UI flow and helps document how navigation remains consistent between related pages.

Frontend Evidence 3



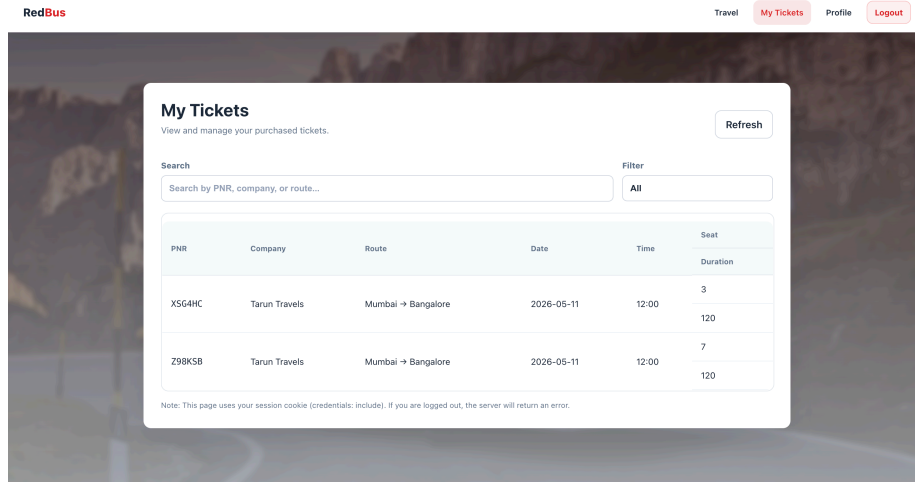
This image is used to demonstrate another stage in the user journey, showing that the interface behavior is state-driven rather than static.

Frontend Evidence 4



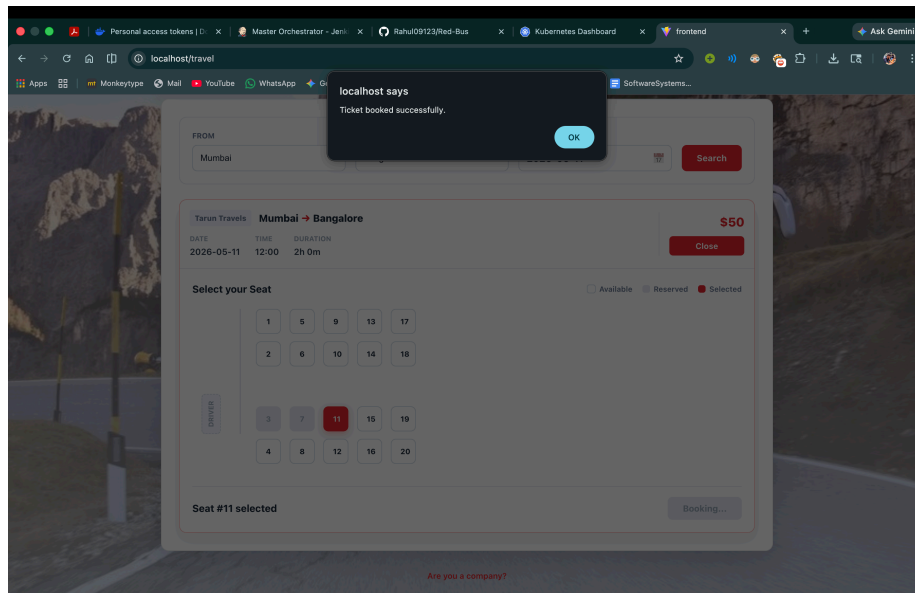
This screenshot captures an additional functional state, supporting the claim that the frontend is split into multiple role or action-specific screens.

Frontend Evidence 5



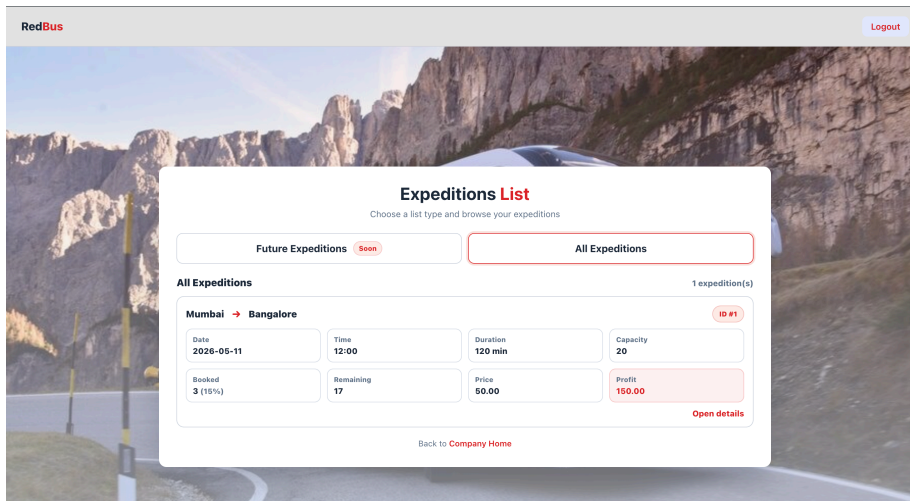
This figure supports route-guarded UI behavior by showing a separate context in the interface flow.

Frontend Evidence 6



This page helps explain the modular page architecture where each user action transitions into a dedicated view component.

Frontend Evidence 7



This final frontend evidence image complements the route and layout explanation by showing another operational state in the same application.

7. Database and Persistence Strategy

PostgreSQL is used as the persistence backbone with initialized databases/scripts under `db/init`. The project follows a domain-isolation mindset where different services manage their own data concerns rather than sharing one tightly coupled schema. This improves modularity and lowers cross-service migration risk.

Keeping service-owned persistence boundaries also makes long-term scaling and refactoring safer, because one domain can change its schema with minimal blast radius.

8. CI/CD and Release Automation

Jenkins is used as the automation engine with an orchestrator pipeline and per-service pipelines. The orchestrator pipeline clones the repository, triggers service builds in parallel, updates deployment manifests with commit-based image tags, and applies Kubernetes manifests in sequence.

Service-level Jenkinsfiles build code (Maven/npm), build Docker images, and push images to Docker Hub using stored credentials. This setup demonstrates practical CI/CD behavior where changes propagate from Git history into deployable artifacts and then into cluster updates.

The result is a repeatable release path that reduces manual intervention and supports incremental delivery.

CI/CD Evidence 1

This screenshot shows the 'Create Expedition' form in the RedBus application. The form is set against a background image of a white bus on a road. The form fields are as follows:

Departure City		Arrival City	
Bangalore		Hyderabad	
Date: 12/05/2026		Time: 10:00	
Price: 20	Duration (min): 90	Capacity: 42	

A red button labeled 'Create Expedition' is located below the form fields. Below the button, a message states 'Expedition created successfully.' and a link 'Back to Company Home' is provided. A summary section below the message displays the expedition details:

From	To
Bangalore	Hyderabad
Date: 2026-05-12	Time: 10:00
Price: 20	Duration: 90 min
Capacity: 42	

This screenshot represents pipeline-related activity and supports the automated build/deploy narrative described above.

CI/CD Evidence 2

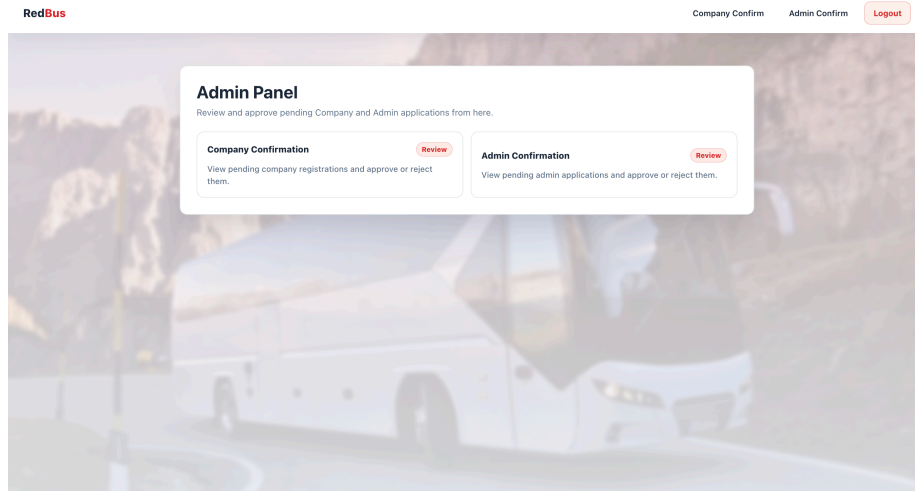
This screenshot shows the 'Expeditions List' page in the RedBus application. The page has a header with the RedBus logo and a 'Logout' button. The main content area is titled 'Expeditions List' and includes a subtitle 'Choose a list type and browse your expeditions'. There are two tabs: 'Future Expeditions' and 'All Expeditions'. The 'All Expeditions' tab is selected, showing a list of 2 expeditions. The first expedition is from Mumbai to Bangalore, and the second is from Bangalore to Hyderabad. Each expedition card displays the following details:

Mumbai → Bangalore			
Date: 2026-05-11	Time: 12:00	Duration: 120 min	Capacity: 20
Booked: 3 (15%)	Remaining: 17	Price: 50.00	Profit: 150.00

The second expedition card follows a similar structure for the Bangalore to Hyderabad route.

This image provides additional evidence of multi-step automation in the release flow.

CI/CD Evidence 3



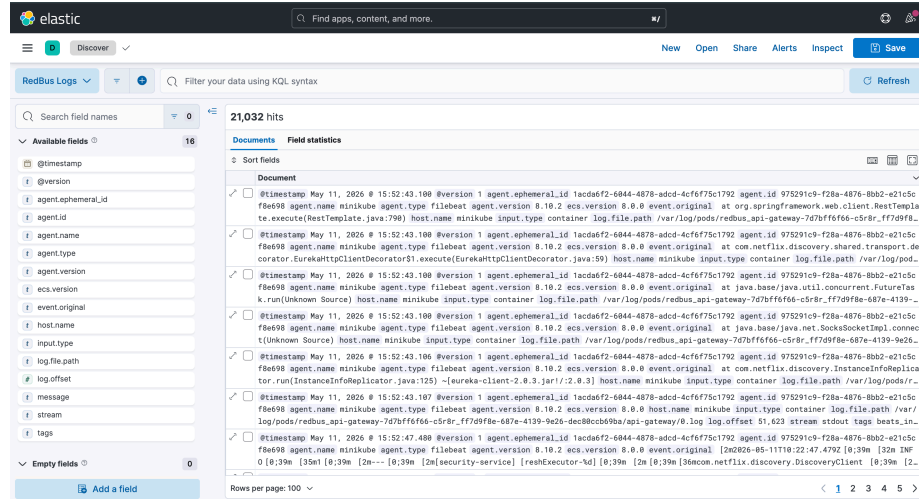
This figure supports the discussion that repository updates can be translated into deployable container artifacts.

9. Containerization and Kubernetes Operations

The project supports both local and cluster execution. Docker Compose gives a fast local integration environment, while Kubernetes manifests provide orchestrated deployment for realistic runtime behavior.

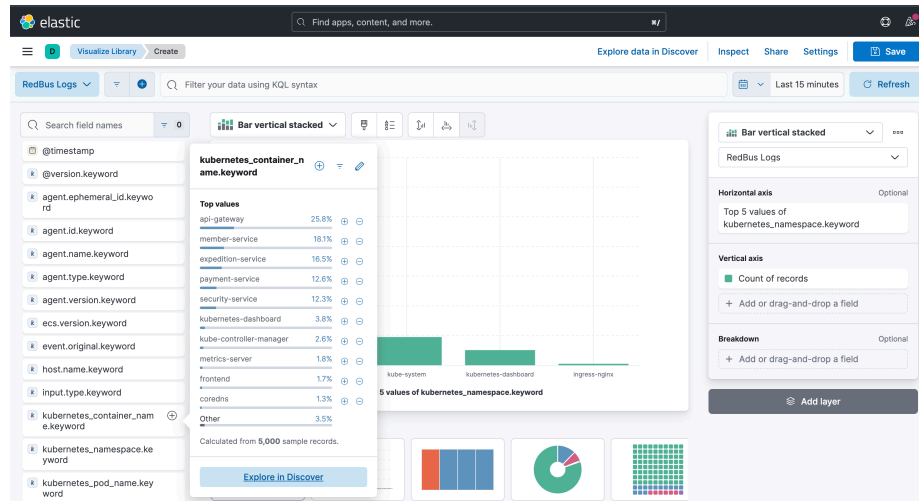
In Kubernetes, each service is deployed with probes and resource definitions. Readiness probes ensure traffic goes only to healthy pods; liveness probes recover failed containers; startup probes protect slower boots from false restarts. This is a key production characteristic, because resilience is built into deployment configuration rather than handled ad hoc.

Deployment Evidence 1



This screenshot is used as evidence of containerized runtime/deployment state and supports the discussion of orchestrated operations.

Deployment Evidence 2



This figure further demonstrates that the system runs through managed deployment resources instead of manual process execution.

10. Scalability, Reliability, and Self-Healing

Horizontal Pod Autoscaler manifests define CPU-based autoscaling for the frontend, gateway, and backend services. Even with conservative min/max values, this proves the ability to scale based on runtime pressure instead of fixed replica assumptions.

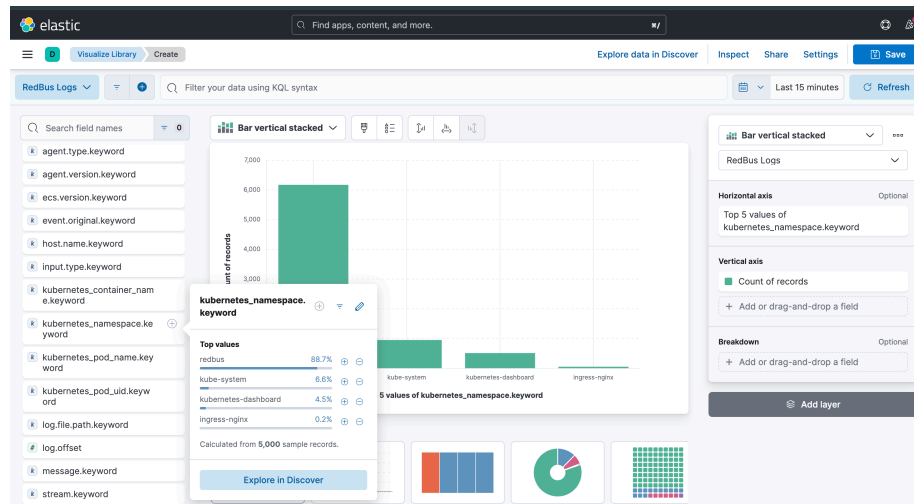
Together with Kubernetes restarts and probe-driven health checks, the platform achieves self-healing behavior expected in modern distributed deployments. Faults in one service do not automatically crash the entire platform, and failing pods can be replaced without full-system downtime.

11. Monitoring and Logging (ELK)

Observability is implemented as a full ELK pipeline. Filebeat runs as a DaemonSet so every node contributes logs. Logstash parses and enriches logs, including Kubernetes context fields. Elasticsearch stores indexed logs for search and correlation. Kibana provides the visualization layer, and is exposed through ingress path routing.

This architecture allows troubleshooting by service, pod, namespace, and time window, which is essential in multi-service systems where root causes are often distributed across components.

Observability Evidence



This screenshot is included to demonstrate runtime visibility and supports the claim that logs are centralized and reviewable through a dashboard interface.

12. Security and Secrets

The platform uses role-aware auth logic in the security service and stores runtime-sensitive values using Kubernetes secrets. Docker registry credentials are also kept as cluster secrets for image pulls. Vault deployment artifacts and Ansible secret tasks are included to demonstrate a security-aware deployment approach.

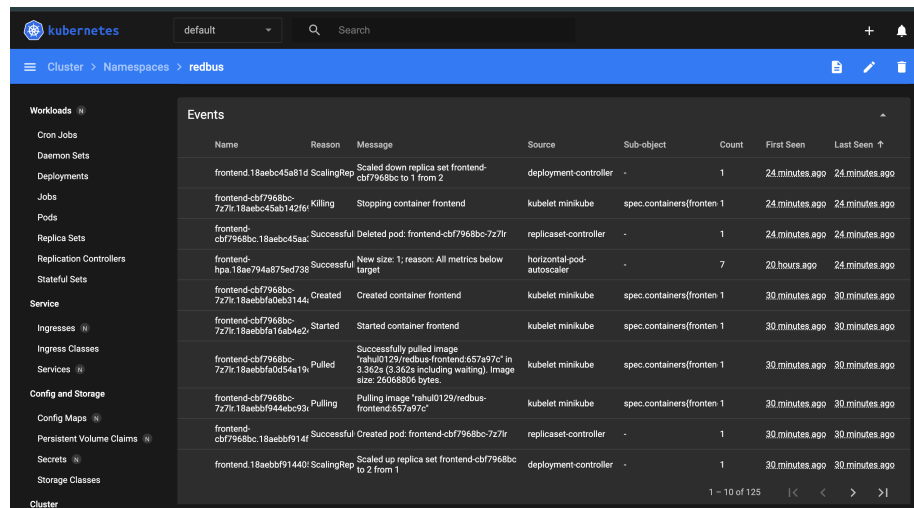
For production hardening, all placeholder/default secrets should be removed and replaced with externally managed, rotated, environment-specific credentials.

13. Configuration Management with Ansible

Ansible playbooks and roles are used to automate environment setup and deployment stages, including namespace setup, secret creation, database deployment, microservice rollout, and verification. This converts many manual operational steps into declarative automation and supports reproducibility across machines or environments.

From an SPE perspective, this is important because infrastructure changes become versioned and reviewable, just like application code.

Configuration Automation Evidence



Name	Reason	Message	Source	Sub-object	Count	First Seen	Last Seen
frontend-18aebc45a81d	ScalingRep	Scaled down replica set frontend-cbf7968bc to 1 from 2	deployment-controller	-	1	24 minutes ago	24 minutes ago
frontend-cbf7968bc-727lr-18aebc45ab142f6	Killing	Stopping container frontend	kubelet minikube	spec.containers(fronten	1	24 minutes ago	24 minutes ago
frontend-cbf7968bc-18aebc45aa	Successful	Deleted pod: frontend-cbf7968bc-727lr	replicaset-controller	-	1	24 minutes ago	24 minutes ago
frontend-hpa-18ae794a875ed738	Successful	New size: 1; reason: All metrics below target	horizontal-pod-autoscaler	-	7	20 hours ago	24 minutes ago
frontend-cbf7968bc-727lr-18aebbf9aeb3144	Created	Created container frontend	kubelet minikube	spec.containers(fronten	1	30 minutes ago	30 minutes ago
frontend-cbf7968bc-727lr-18aebbf9aeb3144	Started	Started container frontend	kubelet minikube	spec.containers(fronten	1	30 minutes ago	30 minutes ago
frontend-cbf7968bc-727lr-18aebbf9aeb3144	Pulled	Successfully pulled image 'rahu0129/redbus-frontend:557a97c' in 3.362s (3.362s including waiting). Image size: 26068806 bytes.	kubelet minikube	spec.containers(fronten	1	30 minutes ago	30 minutes ago
frontend-cbf7968bc-727lr-18aebbf944ebc93	Pulling	Pulling image 'rahu0129/redbus-frontend:557a97c'	kubelet minikube	spec.containers(fronten	1	30 minutes ago	30 minutes ago
frontend-cbf7968bc-18aebbf914f	Successful	Created pod: frontend-cbf7968bc-727lr	replicaset-controller	-	1	30 minutes ago	30 minutes ago
frontend-18aebbf91440	ScalingRep	Scaled up replica set frontend-cbf7968bc to 2 from 1	deployment-controller	-	1	30 minutes ago	30 minutes ago

This image is used to support the configuration-management section, indicating infrastructure/deployment operations are scripted and repeatable.

14. Compliance with Final Project Criteria

This implementation satisfies the core evaluation expectations: version-controlled development, automated CI/CD pipeline behavior, Docker image lifecycle, Kubernetes deployment, and centralized logging with Kibana visibility. It also includes advanced aspects such as HPA and Ansible role-based automation, plus a vault-related security component.

The project is clearly domain-specific (transport ticketing) and not a generic CRUD demo, which aligns with the course recommendation for practical, real-world application domains.

15. Conclusion

RedBus demonstrates an end-to-end software production engineering workflow: modular microservices design, automated build-and-deploy flow, orchestrated runtime management, and centralized operational visibility. The project shows how system design, delivery automation, and runtime operations must be treated as one integrated engineering problem.

This makes the solution suitable for final-project evaluation not only as a working application, but as a complete DevOps-driven production model.